

“MAKALAH TUGAS 3 SISTEM OPERASI”

**Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Operasi**

Dosen : Triawan Adi Cahyanto, M.Kom



**Disusun Oleh:
Reandra Khansa Faadihilah 2410651045**

**UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026**

DAFTAR ISI

BAB I KALKULATOR PARALEL	3
BAB II FILE PROCESSING	6
BAB III PRODUCER DAN CONSUMER.....	10

BAB I

KALKULATOR PARALEL

Pada tugas satu kali ini kita membuat kalkulator paralel sederhana yang dapat menghitung penjumlahan, pengurangan, perkalian dan pembagian dengan operasi dilakukan secara bersamaan (paralel) yang menggunakan beberapa thread.

PRAKTIK

Pertama membuat file `calc_parallel.c` menggunakan text editor `gedit`

```
^ ~/S03/TUGAS > gedit calc_parallel.c @ 21:19
```

```
#include <stdio.h>
#include <pthread.h>

typedef struct {
    double a;
    double b;
    double hasil;
    char operasi;
} DataOperasi;

// Fungsi yang dijalankan oleh setiap thread
void* hitung(void* arg) {
    DataOperasi* data = (DataOperasi*) arg;

    switch (data->operasi) {
        case '+':
            data->hasil = data->a + data->b;
            break;
        case '-':
            data->hasil = data->a - data->b;
            break;
        case '*':
            data->hasil = data->a * data->b;
            break;
        case '/':
            if (data->b == 0)
                printf("Error: Pembagian dengan nol!\n");
            else
                data->hasil = data->a / data->b;
            break;
    }
    pthread_exit(NULL);
}

int main() {
    pthread_t thread[4];
    DataOperasi data[4];
    char operasi[] = {'+', '-', '*', '/'};

    printf("===== KALKULATOR PARALEL =====\n");
    printf("Masukkan angka pertama: ");
    scanf("%lf", &data[0].a);
    printf("Masukkan angka kedua: ");
    scanf("%lf", &data[0].b);

    // Duplikasi nilai untuk setiap thread
    for (int i = 0; i < 4; i++) {
        data[i].a = data[0].a;
        data[i].b = data[0].b;
        data[i].operasi = operasi[i];
        pthread_create(&thread[i], NULL, hitung, &data[i]);
    }

    for (int i = 0; i < 4; i++) {
        pthread_join(thread[i], NULL);
    }

    printf("\n===== HASIL PERHITUNGAN =====\n");
    printf("%lf + %lf = %lf\n", data[0].a, data[0].b, data[0].hasil);
    printf("%lf - %lf = %lf\n", data[1].a, data[1].b, data[1].hasil);
    printf("%lf * %lf = %lf\n", data[2].a, data[2].b, data[2].hasil);
    if (data[3].b != 0)
        printf("%lf / %lf = %lf\n", data[3].a, data[3].b, data[3].hasil);

    printf("=====\n");
    return 0;
}
```

Compile dan jalankan program :

```
^ ~/S03/TUGAS > gcc calc_parallel.c -o calc_parallel
^ ~/S03/TUGAS > ./calc_parallel
== KALKULATOR PARALEL ==
Masukkan angka pertama: 15
Masukkan angka kedua: 3

== HASIL PERHITUNGAN ==
15.00 + 3.00 = 18.00
15.00 - 3.00 = 12.00
15.00 * 3.00 = 45.00
15.00 / 3.00 = 5.00
=====
^ ~/S03/TUGAS > |
```

Penjelasan :

#include <stdio.h>

#include <pthread.h>

- Berfungsi memanggil library **stdio.h** dengan fungsi menerima operasi input/output, kemudian **pthread.h** yang berisi **pthread_create & pthread_join()** dengan fungsi membuat dan mengelola thread.

typedef struct {

double a;

double b;

double hasil;

char operasi;

} DataOperasi;

- Berisi 2 angka struktur data a, b dengan hasil operasi dan jenis operasi (+,-,*,/)

void* hitung(void* arg) {

DataOperasi* data =

(DataOperasi*) arg;

- Fungsi yang akan dijalankan setiap thread

```

switch (data->operasi) {
    case '+': data->hasil = data->a + data->b; break;
    case '-': data->hasil = data->a - data->b; break;
    case '*': data->hasil = data->a * data->b; break;
    case '/':
        if (data->b == 0) printf("Error: Pembagian dengan nol!\n");
        else data->hasil = data->a / data->b;
        break;
}

```

- Mengecek simbol operasi dan mengerjakan sesuai tugasnya

```
pthread_exit(NULL);
```

- Menghentikan thread setelah selesai melakukan proses

```
pthread_t thread[4];
```

```
DataOperasi data[4];
```

```
char operasi[] = {'+', '-', '*', '/'};
```

- Menyimpan 4 thread dan 4 paket data untuk setiap operasi

```
printf("Masukkan angka pertama: ");
```

```
scanf("%lf", &data[0].a);
```

- Mencetak Masukkan angka pertama dan juga meminta input dari user

```
for (int i = 0; i < 4; i++) {
```

```
    data[i].a = data[0].a;
```

```
    data[i].b = data[0].b;
```

```
    data[i].operasi = operasi[i];
```

```
    pthread_create(&thread[i], NULL, hitung, &data[i]);
```

```
}
```

- Membuat 4 thread dengan setiap thread diberi operasi yang berbeda

```
for (int i = 0; i < 4; i++)
```

```
    pthread_join(thread[i], NULL)
```

- Berfungsi menunggu child selesai bekerja

```
printf("%.2lf + %.2lf = %.2lf\n", data[0].a, data[0].b, data[0].hasil);
```

- Menampilkan hasil dari semua thread

BAB II FILE PROCESSING

Pada tugas kedua kita membuat program dengan menghitung jumlah kata dalam file teks, kemudian membandingkan waktu eksekusi antara single thread multi thread

PRAKTIK

Pertama membuat file `_proses.c` dengan text editor gedit

^ ~/S03/TUGAS > gedit file_proses.c c -gcc © 21:55

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <ctype.h>
5 #include <time.h>
6
7 typedef struct {
8     char *buffer;
9     long start;
10    long end;
11    int count;
12 } ThreadData;
13
14 int count_words(char *buf, long start, long end) {
15     int in_word = 0, count = 0;
16     for (long i = start; i < end; i++) {
17         if (isspace(buf[i])) {
18             if (in_word) {
19                 count++;
20                 in_word = 0;
21             }
22         } else {
23             in_word = 1;
24         }
25     }
26     if (in_word) count++;
27     return count;
28 }
29
30 void* thread_count(void* arg) {
31     ThreadData* data = (ThreadData*) arg;
32     data->count = count_words(data->buffer, data->start, data->end);
33     pthread_exit(NULL);
34 }
35
36 int main() {
37     FILE *f = fopen("text.txt", "r");
38     if (!f) {
39         printf("File tidak ditemukan!\n");
40         return 1;
41     }
42 }
```

```

41 }
42
43 fseek(f, 0, SEEK_END);
44 long size = ftell(f);
45 rewind(f);
46
47 char *buffer = malloc(size + 1);
48 fread(buffer, 1, size, f);
49 buffer[size] = '\0';
50 fclose(f);
51
52 // Single-thread timing
53 clock_t start = clock();
54 int single = count_words(buffer, 0, size);
55 clock_t end = clock();
56 double single_time = (double)(end - start) / CLOCKS_PER_SEC;
57
58 // Multi-thread timing
59 ThreadData td[2];
60 pthread_t t[2];
61 long mid = size / 2;
62
63 td[0].buffer = buffer; td[0].start = 0; td[0].end = mid;
64 td[1].buffer = buffer; td[1].start = mid; td[1].end = size;
65
66 start = clock();
67 for (int i = 0; i < 2; i++)
68     pthread_create(&t[i], NULL, thread_count, &td[i]);
69 for (int i = 0; i < 2; i++)
70     pthread_join(t[i], NULL);
71 end = clock();
72
73 int multi = td[0].count + td[1].count;
74 double multi_time = (double)(end - start) / CLOCKS_PER_SEC;
75
76 printf("Jumlah kata (single-thread): %d | Waktu: %.6f s\n", single, single_time);
77 printf("Jumlah kata (multi-thread) : %d | Waktu: %.6f s\n", multi, multi_time);
78
79 free(buffer);
80 return 0;
81 }
82

```

Kemudian Compile Program

```

^ ~/S03/TUGAS > gcc file_proses.c -o file_proses -pthread
./file_proses
Masukkan nama file teks: test.txt

=== HASIL PEMBACAAN FILE ===
Jumlah kata (single-thread): 0 | Waktu: 0.000002 s
Jumlah kata (multi-thread) : 0 | Waktu: 0.000670 s
^ ~/S03/TUGAS > |

```

Penjelasan :

Pthread.h :

- Berfungsi membuat thread

Ctype.h :

Ispace :

- Mengecek spasi dan huruf

time.h

- Mengukur waktu proses

```
typedef struct {
```

```
    char *buffer;
```

```
    long start;
```

```
    long end;
```

```
    int count;
```

```
} ThreadData;
```

- `buffer` = isi file,
- `start` & `end` = awalan dan akhiran bagian file yang dihitung
- `count` = jumlah kata dari hasil hitungan thread itu.

```
int count_words(char *buf, long start, long end) {
```

```
    int in_word = 0, count = 0;
```

```
    for (long i = start; i < end; i++) {
```

```
        if (isspace(buf[i])) {
```

```
            if (in_word) { count++; in_word = 0; }
```

```
        } else {
```

```
            in_word = 1;
```

```
        }
```

```
    }
```

```
    if (in_word) count++;
```

```
    return count;
```

```
}
```

- Berfungsi menghitung kata manual, jika menemukan spasi kata sudah selesai, jika bukan spasi maka masih di dalam kata

```
void* thread_count(void* arg) {
```

```
    ThreadData* data = (ThreadData*) arg;
```

```
    data->count = count_words(data->buffer, data->start, data->end);
```

```
    pthread_exit(NULL);
```

```
}
```

- Thread berfungsi memanggil `count_words()` kemudian membuat bagian filenya masing-masing.


```
FILE *f = fopen(filename, "r");
```

```
if (!f) {
```

```
    printf("File '%s' tidak ditemukan!\n", filename);
```

```
    return 1;
```

```
}
```

Berfungsi membuka file yang user input, jika tidak ada maka gagal “File tidak ditemukan”

```
fseek(f, 0, SEEK_END);
```

```
long size = ftell(f);
```

```
rewind(f);
```

- Mencari tahu ukuran file seberapa banyak data yang dibaca

```
char *buffer = malloc(size + 1);
```

```
fread(buffer, 1, size, f);
```

```
buffer[size] = '\0';
```

- Membaca file ke dalam variabel buffer

```
clock_t start = clock();
```

```
int single = count_words(buffer, 0, size);
```

```
clock_t end = clock();
```

- Menghitung semua kata dan juga mengukur waktunya

```
long mid = size / 2;
```

```
td[0].start = 0; td[0].end = mid;
```

```
td[1].start = mid; td[1].end = size;
```

- Membagi 2 bagian file agar 2 thread bekerja secara bersama.

```
pthread_create(&t[0], NULL, thread_count, &td[0]);
```

```
pthread_create(&t[1], NULL, thread_count, &td[1]);
```

```
pthread_join(t[0], NULL);
```

```
pthread_join(t[1], NULL);
```

- Membuat 2 thread dan menunggu keduanya selesai, masing-masing menghitung kata disetengah file.

```
int multi = td[0].count + td[1].count;
```

- Menggabung hasil dari dua thread menjadi total jumlah kata keseluruhan

```
printf("Jumlah kata (single-thread): %d | Waktu: %.6f s\n", single, single_time);
```

```
printf("Jumlah kata (multi-thread) : %d | Waktu: %.6f s\n", multi, multi_time);
```

- Mencetak hasil dan waktu eksekusi dan membandingkannya

BAB III

PRODUCER DAN CONSUMER

Pada tugas ke-3 kali ini kita membuat program dengan membuat 2 thread (Producer dan Consumer) saling bekerja sama memakai buffer dan mutex, agar data aman dan tidak tumpang tindih.

PRAKTIK

Langsung saja membuat file dengan nama producer_consumer.c menggunakan text editor gedit :

```
^ ~/S03/TUGAS > gedit producer_consumer.c c -gcc 22:09
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4 #include <unistd.h>
5
6 #define MAX_BUFFER 10 // Kapasitas maksimum buffer
7 #define TOTAL_ITEM 20 // Total item yang akan diproduksi
8
9 int buffer[MAX_BUFFER];
10 int count = 0; // Jumlah item dalam buffer
11 pthread_mutex_t lock; // Mutex untuk mengamankan area kritis
12 pthread_cond_t not_full, not_empty; // Kondisi buffer penuh / kosong
13
14 // Fungsi thread producer
15 void* producer(void* arg) {
16     for (int i = 0; i < TOTAL_ITEM; i++) {
17         pthread_mutex_lock(&lock);
18
19         // Tunggu jika buffer penuh
20         while (count == MAX_BUFFER) {
21             pthread_cond_wait(&not_full, &lock);
22         }
23
24         // Tambah item ke buffer
25         int item = rand() % 100; // Angka random 0-99
26         buffer[count++] = item;
27         printf("[Producer] menghasilkan item %d (isi buffer: %d)\n", item, count);
28
29         // Sinyal ke consumer bahwa buffer tidak kosong lagi
30         pthread_cond_signal(&not_empty);
31         pthread_mutex_unlock(&lock);
32
33         usleep(100000); // Simulasi jeda produksi (0.1 detik)
34     }
35     pthread_exit(NULL);
36 }
37
38 // Fungsi thread consumer
39 void* consumer(void* arg) {
40     for (int i = 0; i < TOTAL_ITEM; i++) {
41         pthread_mutex_lock(&lock);
42
43         // Tunggu jika buffer kosong
44         while (count == 0) {
45             pthread_cond_wait(&not_empty, &lock);
46         }
47
48         // Ambil item dari buffer
49         int item = buffer[--count];
50         printf("[Consumer] mengambil item %d (isi buffer: %d)\n", item, count);
51
52         // Sinyal ke producer bahwa buffer tidak penuh lagi
53         pthread_cond_signal(&not_full);
54         pthread_mutex_unlock(&lock);
55
56         usleep(150000); // Simulasi waktu pemrosesan (0.15 detik)
57     }
58     pthread_exit(NULL);
59 }
60
61 int main() {
62     pthread_t prod, cons;
63
64     // Inisialisasi mutex dan kondisi
65     pthread_mutex_init(&lock, NULL);
66     pthread_cond_init(&not_full, NULL);
67     pthread_cond_init(&not_empty, NULL);
68
69     printf("=== SIMULASI PRODUCER-CONSUMER ===\n\n");
70
71     // Buat thread producer dan consumer
72     pthread_create(&prod, NULL, producer, NULL);
73     pthread_create(&cons, NULL, consumer, NULL);
74
75     // Tunggu kedua thread selesai
76     pthread_join(prod, NULL);
77     pthread_join(cons, NULL);
78
79     // Bersihkan resource
80     pthread_mutex_destroy(&lock);
81     pthread_cond_destroy(&not_full);
82     pthread_cond_destroy(&not_empty);
83
84     printf("\n=== Program selesai ===\n");
85     return 0;
86 }
```

Kemudian Compile programnya

```
(org.gnome.gedit:44791): libenchant-WARNING **: 22:09:41.689: broker.val
^ ~/S03/TUGAS > gcc producer_consumer.c -o producer_consumer -pthread
./producer_consumer
```

=== SIMULASI PRODUCER-CONSUMER ===

```
[Producer] menghasilkan item 83 (isi buffer: 1)
[Consumer] mengambil item 83 (sis buffer: 0)
[Producer] menghasilkan item 86 (isi buffer: 1)
[Consumer] mengambil item 86 (sis buffer: 0)
[Producer] menghasilkan item 77 (isi buffer: 1)
[Consumer] mengambil item 77 (sis buffer: 0)
[Producer] menghasilkan item 15 (isi buffer: 1)
[Producer] menghasilkan item 93 (isi buffer: 2)
[Consumer] mengambil item 93 (sis buffer: 1)
[Producer] menghasilkan item 35 (isi buffer: 2)
[Consumer] mengambil item 35 (sis buffer: 1)
[Producer] menghasilkan item 86 (isi buffer: 2)
[Producer] menghasilkan item 92 (isi buffer: 3)
[Consumer] mengambil item 92 (sis buffer: 2)
[Producer] menghasilkan item 49 (isi buffer: 3)
[Consumer] mengambil item 49 (sis buffer: 2)
[Producer] menghasilkan item 21 (isi buffer: 3)
[Producer] menghasilkan item 62 (isi buffer: 4)
[Consumer] mengambil item 62 (sis buffer: 3)
[Producer] menghasilkan item 27 (isi buffer: 4)
[Consumer] mengambil item 27 (sis buffer: 3)
[Producer] menghasilkan item 90 (isi buffer: 4)
[Producer] menghasilkan item 59 (isi buffer: 5)
[Consumer] mengambil item 59 (sis buffer: 4)
[Producer] menghasilkan item 63 (isi buffer: 5)
[Consumer] mengambil item 63 (sis buffer: 4)
[Producer] menghasilkan item 26 (isi buffer: 5)
[Producer] menghasilkan item 40 (isi buffer: 6)
[Consumer] mengambil item 40 (sis buffer: 5)
[Producer] menghasilkan item 26 (isi buffer: 6)
[Consumer] mengambil item 26 (sis buffer: 5)
[Producer] menghasilkan item 72 (isi buffer: 6)
[Producer] menghasilkan item 36 (isi buffer: 7)
[Consumer] mengambil item 36 (sis buffer: 6)
[Consumer] mengambil item 72 (sis buffer: 5)
[Consumer] mengambil item 26 (sis buffer: 4)
[Consumer] mengambil item 90 (sis buffer: 3)
[Consumer] mengambil item 21 (sis buffer: 2)
[Consumer] mengambil item 86 (sis buffer: 1)
[Consumer] mengambil item 15 (sis buffer: 0)
```

=== Program selesai ===

^ ~/S03/TUGAS > |

PENJELASAN :

#define MAX_BUFFER 10

- Membuat kapasitas maksimal penyimpanan

#define TOTAL_ITEM 20

- Jumlah item yang akan diproduksi dan dikonsumsi

```
int buffer[MAX_BUFFER];
```

```
int count = 0;
```

```
pthread_mutex_t lock;
```

```
pthread_cond_t not_full, not_empty;
```

- Buffer : Berfungsi menyimpan data sementara
- Count : Menghitung berapa item yang berada di buffer sekarang
- Lock : Kunci thread agar tidak terjadi tumpang tindih.

Not_full & Not_empty

- Memberi sinyal kapan buffer penuh atau kosong

```
while (count == MAX_BUFFER)
```

```
pthread_cond_wait(&not_full, &lock);
```

- Jika buffer penuh, produser harus menunggu sampai ada tempat kosong

```
int item = rand() % 100;
```

```
buffer[count++] = item;
```

```
printf("[Producer] menghasilkan item %d (isi buffer: %d)\n", item, count);
```

```
pthread_cond_signal(&not_empty);
```

- Membuat angka acak kemudian masukkan ke buffer, lalu memberi tahu consumer jika buffer tidak kosong

```
while (count == 0)
```

```
pthread_cond_wait(&not_empty, &lock);
```

- Jika buffer kosong, maka consumer harus menunggu sampai ada data

```
int item = buffer[--count];
```

```
printf(" [Consumer] mengambil item %d (sis buffer: %d)\n", item, count);
```

```
pthread_cond_signal(&not_full)
```

- Berfungsi mengambil item terakhir dari buffer, kemudian mengasih sinyal ke producer jika buffer tidak penuh

```
pthread_mutex_lock(&lock)
```

```
pthread_mutex_unlock(&lock)
```

- Ketika setiap mengakses buffer, maka harus dikunci terlebih dahulu supaya tidak bentrok dengan thread yang lain